Advanced Programming
Python

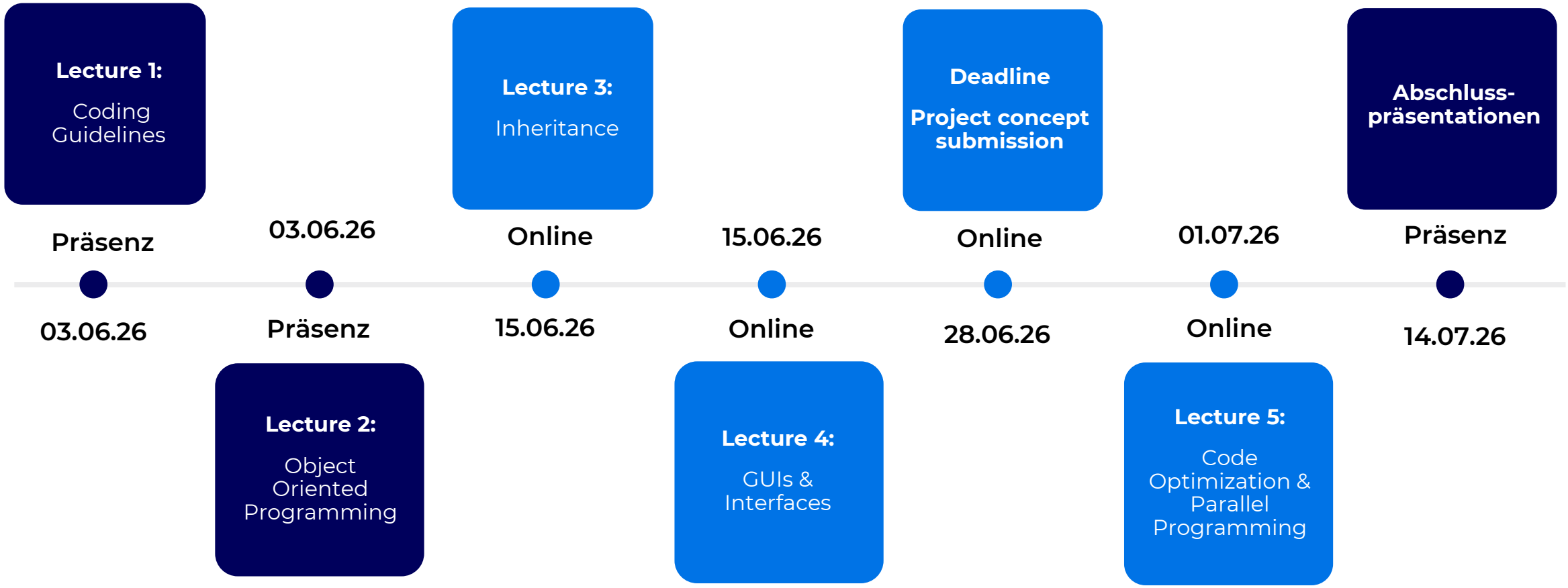
15.06.2026

Marius Kiskemper
Marius.Kiskemper@atos.ai

Atos

Organisatorisches

Termine



Organisatorisches

Prüfungsleistung

- Gruppen- oder Einzelabgabe
  - Falls Gruppen: Maximal 3 Personen
- Ziel: Jede Gruppe sucht sich einen beliebigen Use Case, der durch ein Python-Projekt umgesetzt wird
- Zu Beginn wird ein Code-Konzept abgegeben (fließt auch in die Bewertung ein)
- Das fertige Python-Projekt wird am Ende abgegeben und bewertet
- Zudem wird am Ende eine Abschlusspräsentation gehalten, die bewertet wird

Organisatorisches

Prüfungsleistung

Teil 1: **Code-Konzept** (Use-Case-Beschreibung)

Deadline: 28.06.26

- Es soll dargestellt werden, welcher Use-Case im Rahmen des Code-Projekts umgesetzt wird
- Use-Case kann thematisch alles sein → einzige Vorgabe: irgendeine Art Klassenkonstrukt
  - Fahrzeugvermietung, Restaurantsimulation (z.B. Reservierungssoftware), Banksimulation (z.B. Kreditwürdigkeit), Machine Learning – Pipeline, …
- Konzeptionelles Beschreiben des Use-Cases
  - Welche Klassen werden genutzt? Wie interagieren die Objekte der Klassen? Was sind wichtige Attribute und Methoden?
  - Diese Ideen sind nicht unveränderbar → in der Endlösung können auch noch Klassen hinzukommen oder entfernt werden
    - Die Abgabe vom 28.06. wird aber in sich geschlossen bewertet
- Wie werden die Coding-Guidelines umgesetzt?
- Aufwandsschätzung → Wie sorgt man vorher für eine effiziente Implementierung?

Abgabe: Ein PDF (Deadline: 28.06.) mit allen oben genannten Inhalten

→ bis zu **15 Punkte** (für alle Gruppenmitglieder gleich (außer individuell angegeben))

Kurzvorstellung eurer Idee in der Vorlesung am 01.07.

Organisatorisches

Prüfungsleistung - Ergänzung

Teil 2: **Implementierung des Projekts**

- Es soll lauffähiger Code abgegeben werden
- Bewertet werden dabei:
  - Inhaltliche Richtigkeit (macht der Code was er soll?)
  - Test-Robustheit (ist der Code stabil gegenüber verschiedenen Test-Eingaben?)
  - Beachtung der Python Best-Practices
  - Code-Nachvollziehbarkeit (z.B. Kommentare)

Abgabe: Ein GitHub-Link, der das gesamte Projekt enthält, sodass ich es sofort bei mir ausführen kann (inkl. requirements.txt und Ausführ-Anweisungen)

→ **Commit-History wird bewertet** → es wird erwartet, dass viele kleine Arbeitspakete (inkl. Branching etc.) committed werden

→ bis zu **25 Punkte** (für alle Gruppenmitglieder gleich (außer individuell angegeben))

Abgabe-Deadline: 14.07.26

Organisatorisches

Prüfungsleistung

Teil 3: **Präsentation des Projekts**

- Es soll das implementierte Projekt präsentiert werden (inkl. **Live-Demo**)
- Zielgruppe: CEO & CTO (Business- und Technik-Sicht)
- Bewertet werden dabei:
  - Präsentationsformat (Wie für einen Kunden, den man von seinem Produkt überzeugen möchte)
  - Nachvollziehbarkeit & roter Faden
  - Rückblick (Was hat gut funktioniert, was nicht und warum?)
  - Anspruch der Lösung
  - Kompetenz bei Beantwortung der gestellten Fragen
    - Jeder muss jede Frage beantworten können → Antworten können individuell bewertet werden

Abgabe: Eine PDF-Datei, die die gehaltene Präsentation beinhaltet

→ bis zu **20 Punkte** (Grundpunktzahl für alle Gruppenmitglieder gleich, Fragen evtl. individuell)

Abgabe-Deadline: 14.07.25

Recap

Recap

- DRY, Single Responsibility, Open Closed
- Planning the implementation
- Code formatting (autopep8, pycodestyle)
- → Not mandatory, but recommended

Recap

- Classes and their instances are used to organize and structure data and their manipulations
  - init-Method used to set attributes
  - instances used to call the functional methods
- Different argument types
- Call by value vs. Call by reference
- Pure function vs. Modifier

Recap

Call by value

```
print("call by value:")

cities_2 = "Mannheim, Berlin, Hamburg"


def add_city_2(city_str):
    city_str += ", Frankfurt"
    return city_str


print("Before function call: ", cities_2)
print("Inside function call: ", add_city_2(cities_2))
print("After function call: ", cities_2)
```

```
call by value:
Before function call:  Mannheim, Berlin, Hamburg
Inside function call:  Mannheim, Berlin, Hamburg, Frankfurt
After function call:  Mannheim, Berlin, Hamburg
```

Call by reference

```
print("Call by reference:")

cities = ["Mannheim", "Berlin", "Hamburg"]


def add_city(city_list):
    city_list.append("Frankfurt")
    return city_list


print("Before function call: ", cities)
print("Inside function call: ", add_city(cities))
print("After function call: ", cities)
```

```
Call by reference:
Before function call:  ['Mannheim', 'Berlin', 'Hamburg']
Inside function call:  ['Mannheim', 'Berlin', 'Hamburg', 'Frankfurt']
After function call:  ['Mannheim', 'Berlin', 'Hamburg', 'Frankfurt']
```


Atos

Recap

Pure Function

```
def add_time(t1, t2):
    sum = Time()
    sum.hour = t1.hour + t2.hour
    sum.minute = t1.minute + t2.minute
    sum.second = t1.second + t2.second
    if sum.second >= 60:
        sum.second -= 60
        sum.minute += 1
    if sum.minute >= 60:
        sum.minute -= 60
        sum.hour += 1
    return sum


start_time = Time(9, 45, 10)
duration = Time(2, 30, 15)

print("Before function call: ", start_time, duration)
print("Result of function call: ", add_time(start_time, duration))
print("After function call: ", start_time, duration)
```

```
Before function call:  09:45:10 02:30:15
Result of function call:  12:15:25
After function call:  09:45:10 02:30:15
```

Modifier Function

```
def increment(time, seconds):
    time.second += seconds
    while seconds > 60:
        if time.second >= 60:
            time.second -= 60
            time.minute += 1
            seconds -= 60

    while time.minute >= 60:
        if time.minute >= 60:
            time.minute -= 60
            time.hour += 1
    return time


start_time = Time(9, 45, 10)

print("Before function call: ", start_time)
print("Result of function call: ", increment(start_time, 1000))
print("After function call: ", start_time)
```

```
Before function call:  09:45:10
Result of function call:  10:01:50
After function call:  10:01:50
```

Recap

- Different argument types

```python
def send_message(message, recipient, *args, copy_to=None, second_copy_to=None):

    print(f"Sending the message {message} to the recipient {recipient}")

    if copy_to is not None:
        print("Additional copy sent to: ", copy_to)

    if second_copy_to is not None:
        print("Additional copy sent to: ", second_copy_to)

    if args:
        print("Attachments contain the following files: ", *args)

    return "Done"
```

```python
# minimum
print("1:")
send_message("Hello World", "Björn")
# use *args
print("2:")
send_message("Hello World", "Björn", "Hulkens", "PDF1",
             "Excel1", "PPT1", second_copy_to="Peter")
# use only positional and keywords
print("3:")
send_message("Hello World", "Hulkens", copy_to="Björn", second_copy_to="Peter")
# wrong order
print("4:")
send_message("Hello World", "Björn", copy_to="Hulkens", second_copy_to="Peter", "PDF1", "Excel2", "PPT2")
```

```
1:
Sending the message Hello World to the recipient Björn
2:
Sending the message Hello World to the recipient Björn
Additional copy sent to:  Peter
Attachments contain the following files:  Hulkens PDF1 Excel1 PPT1
3:
Sending the message Hello World to the recipient Hulkens
Additional copy sent to:  Björn
Additional copy sent to:  Peter
4:
  File "c:\Users\a764135\OneDrive - Eviden\Dozieren_DHBW\2024\Examples\agument_types.py", line 46
    send_message("Hello World", "Björn", copy_to="Hulkens", second_copy_to="Peter", "PDF1", "Excel2", "PPT2")
                                                                                                          ^
SyntaxError: positional argument follows keyword argument
```

Recap

- Add-on: kwargs

```python
def send_message_2(message, recipient, copy_to=None, second_copy_to=None, **kwargs):

    print(f"Sending the message {message} to the recipient {recipient}")

    if copy_to is not None:
        print("Additional copy sent to: ", copy_to)

    if second_copy_to is not None:
        print("Additional copy sent to: ", second_copy_to)

    if kwargs:
        print("Attachments contain the following files: ", kwargs)

    return "Done"
```

```python
# minimum
print("1:")
send_message_2("Hello World", "Hulkens")
# positional and keywords
print("2:")
send_message("Hello World", "Hulkens", copy_to="Björn", second_copy_to="Peter")
# multiple keywords
print("3:")
send_message_2("Hello World", "Björn", copy_to="Hulkens", attach1="PDF1",
               attach2 = "Excel2", date="01.01.23", archived = True)
# positional as keyword
print("4:")
send_message_2("Hello World", copy_to="Björn", recipient="Hulkens", attach1="PDF1", programmieren="langweilig")
# missing positional
print("5:")
send_message_2("Hello World", copy_to="Björn", second_copy_to="Peter", attach1="PDF1", attach2="Excel2")
```

```
1:
Sending the message Hello World to the recipient Hulkens
2:
Sending the message Hello World to the recipient Hulkens
Additional copy sent to:  Björn
Additional copy sent to:  Peter
3:
Sending the message Hello World to the recipient Björn
Additional copy sent to:  Hulkens
Attachments contain the following files:  {'attach1': 'PDF1', 'attach2': 'Excel2', 'date': '01.01.23', 'archived': True}
4:
Sending the message Hello World to the recipient Hulkens
Additional copy sent to:  Björn
Attachments contain the following files:  {'attach1': 'PDF1', 'programmieren': 'langweilig'}
5:
Traceback (most recent call last):
  File "c:\Users\a764135\OneDrive - Eviden\Dozieren_DHBW\2024\Examples\agument_types.py", line 64, in <module>
    send_message_2("Hello World", copy_to="Björn", second_copy_to="Peter", attach1="PDF1", attach2="Excel2")
TypeError: send_message_2() missing 1 required positional argument: 'recipient'
```

Recap / Add-on

Type hints

- A type hint declares what type a parameter is expected to be
- Syntax:
  - Parameter: Type
- Return values:
  - → Type
- optional and not enforced at runtime → ignored during Python execution
- Helpful for collaboration, maintenance & syntax-checking tools

- Special case: Different types possible
  - Declared by „|“

```
def unload_truck(self, kg: float | None = None) -> float:
    """Unload cargo. With no argument the truck is fully unloaded.
```

Recap / Add-on

Type hints

```python
# GOOD — clear types on all parameters and the return
def total_price(quantity: int, unit_price: float) -> float:
    return quantity * unit_price

# BAD — no hints; caller has to guess what's allowed
def total_price(quantity, unit_price):
    return quantity * unit_price

# BAD — misleading hint that doesn't match real usage
def total_price(quantity: str, unit_price: float) -> float:  # quantity isn't a str
```

- Common building blocks:
  - Basic types: int, float, str, bool
  - Collections: list[int], dict[str, float], tuple[int, int]
  - Optional / multiple types: float | None, int | str → the | union
  - Default values combine with hints: def f(x: int = 0) — hint *and* default are separate things
    - Previous rules and differentiation of positional and keyword arguments remain true
  - Fallback pattern: def f(x: float | None = None) → "no value given" handled inside

Recap / Add-on

Type hints

DO‘s

- Annotate public function/method parameters and return types
- Keep hints truthful
- Use X | None when None is a legitimate value.
- Be as specific as useful (list[str] over bare list).
- Use a type checker in your workflow → hints are only obligatory

DONT‘s

- Don't rely on hints for runtime validation
  - → Use if/raise for real input checks.
- Don't write lying or outdated hints
- Don't reach for overly complex types when a simple one communicates the intent.

Agenda

1. Coding Guidelines
2. Object Oriented Programming
3. Inheritance
4. GUIs & Interface
5. Code optimization

01

Inheritance Basics

Atos

General Overview

Inheritance

- Ability to define a new class that is a modified version of an existing class
- Classes with similarities but in parallel also distinguishing features
- Inheritance defines a parent class with general features
  - Each inheriting subclass has the ability to use every defined method of the parent class
  - The subclass is extending the inherited features with new unique functionalities
- Subclasses can override methods from the parent class
- Often the inheritance structure represents the real world structure
  - → easy to understand & design
- Inheritance optimizes code reusability
- Caution: Multiple classes can decrease readability and understandability

Guidelines

Inheritance

- Interfaces should stay the same between parent and child classes
  - E.g. When overriding a method
- → Interface planning is key to successful code maintenance
- Class interface: the attributes and methods of the class
- Method interface: the parameters, return type, general functionality, preconditions (→ the skeleton) of a method
- „clean“ interface: allows caller to do what they want without dealing with unnecessary details and to be intuitive in usage

Defining the inheritance

Implementation

```python
class Person:
    def __init__(self, fname, lname):
        print("Using the Person constructor")
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)


person_1 = Person("John", "Doe")
person_1.printname()
```

- Parent class in the brackets of the child class
- def ExampleTwo(ExampleOne):
  - .....
  - → class ExampleTwo inherits from class ExampleOne

```python
class Student(Person):
    pass


student_1 = Student("Mary", "Sue")
student_1.printname()
```

```
Using the Person constructor
John Doe
Using the Person constructor
Mary Sue
```


Atos

Overriding the init method

Implementation

```python
class Person:
    def __init__(self, fname, lname):
        print("Using the Person constructor")
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)


person_1 = Person("John", "Doe")
person_1.printname()
```

- Subclass has its own __init__()-method
- This gets executed instead of the parent-init
- Super() is used to make a reference to the parent class
  - Super().__init__() uses the init method from the parent class

```python
class Student(Person):
    def __init__(self, fname, lname):
        print("Using the Student constructor")
        super().__init__(fname, lname)


student_1 = Student("Mary", "Sue")
student_1.printname()
```

```
Using the Person constructor
John Doe
Using the Student constructor
Using the Person constructor
Mary Sue
```

Overriding the init method

Implementation

```
class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        print("Using the Student constructor")
        super().__init__(fname, lname)
        self.graduation_year = graduation_year
```

- Modifying the init-method by adding new parameters
- These new parameters are unique to the subclass
- Only objects of the class Student accept the new third parameter
- Setting attributes from both the super-init and the overridden init

```
student_1 = Student("Mary", "Sue", 2019)
student_1.printname()

person_1 = Person("John", "Doe", 2020)
person_1.printname()
```

```
Mary Sue
Traceback (most recent call last):
  File "c:\Users\a764135\..Dokumente\Dozieren_DHBW\Examples\simple_Inheritance.py",
    person_1 = Person("John", "Doe", 2020)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: Person.__init__() takes 3 positional arguments but 4 were given
```


Atos

02

Methods

Creating new methods

Implementation

- Subclasses can have their own methods

  → subclass is just like any regular class, while having access to attributes and methods from a parent class

- The subclass methods are only usable by an instance of the subclass itself

```python
class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def student_summary(self):
        print(
            f"The student {self.firstname} {self.lastname} "
            f"graduated in {self.graduation_year}")


student_1 = Student("Mary", "Sue", 2019)
student_1.student_summary()

person_1 = Person("John", "Doe")
person_1.student_summary()
```

```
The student Mary Sue graduated in 2019
Traceback (most recent call last):
  File "c:\Users\a764135\..Dokumente\Dozieren_DHBW\Examples\simple_Inheritance.py",
    person_1.student_summary()
    ^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'Person' object has no attribute 'student_summary'
```

Overriding methods

General overview

- Just as in the beginning, every object can call printname
- But when the student calls printname a different result is printed
- Student overrides the method printname
- Every object calls the same method
  - But depending on what class the instance belongs to, it behaves differently

```python
class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def printname(self):
        print(
            f"The student {self.firstname} {self.lastname} "
            f"graduated in {self.graduation_year}")


student_1 = Student("Mary", "Sue", 2019)
student_1.printname()

person_1 = Person("John", "Doe")
person_1.printname()
```

```
The student Mary Sue graduated in 2019
John Doe
```


Atos

Overriding methods

Special methods

```python
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def __str__(self):
        return f"This person is called {self.firstname, self.lastname}"


person_1 = Person("John", "Doe")
print(person_1)
```

```
This person is called John Doe
```

```python
class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def __str__(self):
        return f"The name of this student is {self.firstname, self.lastname}"


student_1 = Student("Mary", "Sue", 2019)
print(student_1)
```

```
The name of this student is Mary Sue
```

→ Always when a method of a subclass has the same name as a method of the parent class, the instance of the subclass executes its own method

→ The method is overridden

Overriding methods

caution

- Overriding can change the methods purpose to something unexpected

→ is not „clean“ as it is not what the user expects

- Only use overriding when helpful and advantageous
  - Example: print_object_info()

```python
class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def printname(self):
        print(f"The name of the graduation year is {self.graduation_year}")

    def __str__(self):
        return f"The name of this student is {self.firstname} {self.lastname}"


person_1 = Person("John", "Doe")
person_1.printname()
student_1 = Student("Mary", "Sue", 2019)
student_1.printname()
```

```
John Doe
The name of the graduation year is 2019
```

03

Super ()

Using super()

In any method

- Convention to use super() in __init__()
- Can also be useful in other methods
- Helpful when you don‘t want to completely override a methods functionality, but extend it

```python
class Person:

    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def speak(self):
        print("I am a person.")


person_1 = Person("John", "Doe")
person_1.speak()
```

```
I am a person.
```

```python
class Student(Person):

    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def speak(self):
        super().speak()
        print("I am a student.")


student_1 = Student("Mary", "Sue", 2019)
student_1.speak()
```

```
I am a person.
I am a student.
```


Atos

Using super()

Addition

- Using self in a super() method makes the **subclass instance behave like an instance of the parent class**
- super() automatically passes the self argument

```python
class Person:

    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def speak(self):
        print(f"my name is {self.firstname} {self.lastname} "
              f"and I am a person.")


person_1 = Person("John", "Doe")
person_1.speak()
```

```
my name is John Doe and I am a person.
```

```python
class Student(Person):

    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def speak(self):
        super().speak()
        print(f"my name is {self.firstname} {self.lastname} "
              f"and I am a student.")


student_1 = Student("Mary", "Sue", 2019)
student_1.speak()
```

```
my name is Mary Sue and I am a person.
my name is Mary Sue and I am a student.
```


Atos

Using super()

Addition

```python
class Person:

    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def speak(self):
        print(f"my name is {self.firstname} {self.lastname} "
              f"and I am a {self.__class__.__name__}.")


person_1 = Person("John", "Doe")
person_1.speak()
```

```
my name is John Doe and I am a Person.
```

- The subclass instance only behaves temporarily as a parent class instance
- → Being able to call the parent class method does not convert the whole type of the instance

```python
class Student(Person):

    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def speak(self):
        super().speak()
        print(f"my name is {self.firstname} {self.lastname} "
              f"and I am a {self.__class__.__name__}.")


student_1 = Student("Mary", "Sue", 2019)
student_1.speak()
```

```
my name is Mary Sue and I am a Student.
my name is Mary Sue and I am a Student.
```


Atos

04

Multi-level inheritance

Atos

Multi-level inheritance

Implementation

- A child class can be the parent class of another inheriting class
- → Nested classes
- Any subclass instance can access all methods and attributes from every parent class
- Many inheritance levels can make readability ans maintainability hard
- → Depth of Inheritance (DIT) measures the maximum inheritance level in a given project

```python
class Person():
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def __str__(self):
        return f"This person is called {self.firstname} {self.lastname}"


class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def print_year(self):
        print("Graduation year is ", self.graduation_year)


class Programmer(Student):
    def write_code(self, exercise):
        print("Writing code to solve the exercise ", exercise)


programmer_1 = Programmer("Progra", "Mierer", 2020)
print(programmer_1)
programmer_1.print_year()
programmer_1.write_code("Implement multi-level inheritance")
```

```
This person is called Progra Mierer
Graduation year is  2020
Writing code to solve the exercise  Implement multi-level inheritance
```


Atos

Multi-level inheritance

Using super()

- Super() calls the parent class of a given class
- But with nested classes a subclass can have multiple parent classes
  - Which one will be referenced by super()?

→ The direct parent class is referenced

```python
class Person():
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def print_name(self):
        print(f"Name of person is {self.firstname, self.lastname}")


class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def print_name(self):
        print(f"Name of student is {self.firstname} {self.lastname}")


class Programmer(Student):
    def super_test(self):
        super().print_name()


programmer_1 = Programmer("Progra", "Mierer", 2020)
programmer_1.super_test()
```

```
Name of student is Progra Mierer
```

05

Abstract classes

Atos

Abstract Base Classes

Parent classes as ABC

- Abstract Base Classes are used to define a skeleton for the subclasses
- Usually no object is created from an Abstract Base Class
- Using abstract methods to dictate methods of the subclasses
- No must use, but can be helpful in larger class constructions

Abstractmethod

General rule

- Once a parent class defines abstractmethods it **can not instantiate an object anymore**
- Using abstractmethods makes the parent class „abstract“ (only used for guiding the implementation)
- In most cases the abstractmethod has no content

```python
from abc import ABC, abstractmethod


class Person(ABC):
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def __str__(self):
        return f"This person is called {self.firstname} {self.lastname}"

    @abstractmethod
    def print_object_info(self):
        pass

    @abstractmethod
    def print_type(self):
        pass


person_1 = Person("John", "Doe")
```

```
    person_1 = Person("John", "Doe")
               ^^^^^^^^^^^^^^^^^^^^^
TypeError: Can't instantiate abstract class Person with abstract methods print_object_info, print_type
```

Abstractmethod

General rule

- Every defined abstractmethod of a parent class **must be implemented / overridden** by the child class
  - Even if the missing method is not called
- Parent class method without @abstractmethod remain the same

```
class Student(Person):

    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def print_object_info(self):
        print("My attributes are:")
        for attr in dir(self):
            if not attr.startswith('__') and not callable(getattr(self, attr)):
                print(attr)


student_1 = Student("Mary", "Sue", 2019)
student_1.print_object_info()
```

```
    student_1 = Student("Mary", "Sue", 2019)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: Can't instantiate abstract class Student with abstract method print_type
```

Abstractmethod

Implementation

- When the abstractmethods are overriden they behave like every other method
- Advantage is that during defining the parent class you can make sure that these methods will be implemented in the subclass
- Access to the remaining methods stays the same

```python
class Student(Person):

    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def print_object_info(self):
        print("My attributes are:")
        for attr in dir(self):
            if not attr.startswith('__') and not callable(getattr(self, attr)):
                print(attr)

    def print_type(self):
        print(f"The type of this object is {self.__class__.__name__}")


student_1 = Student("Mary", "Sue", 2019)
print(student_1)
print("---")
student_1.print_object_info()
print("---")
student_1.print_type()
```

```
This person is called Mary Sue
---
My attributes are:
_abc_impl
firstname
graduation_year
lastname
---
The type of this object is Student
```

Abstractmethod

Using super()

```python
class Person(ABC):

    @abstractmethod
    def print_type(self):
        print("Printing the class name:")
```

- It is possible to execute the abstractmethods using super()
- Useful when something should be repeated in every subclass implementation of this method
- Fails the purpose if the overriding subclass method changes nothing compared to the abstract definition

```python
class Student(Person):

    def print_type(self):
        super().print_type()
        print(f"The type of this object is {self.__class__.__name__}")


student_1 = Student("Mary", "Sue", 2019)
student_1.print_type()
```

```
Printing the class name:
The type of this object is Student
```

06

Inheritance for SOLID

Atos

Inheritance for SOLID principle

Open closed solution

```python
class Person:
    def __init__(self, fname, lname):
        print("Using the Person constructor")
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)


person_1 = Person("John", "Doe")
person_1.printname()
```

- Inheritance solves Open Closed principle because you can create a new subclass with a new unique functionality instead of modifying the existing class
- → Extending the implementation instead of modifying

```python
class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def student_summary(self):
        print(
            f"The student {self.firstname} {self.lastname} "
            f"graduated in {self.graduation_year}")


student_1 = Student("Mary", "Sue", 2019)
student_1.student_summary()

person_1 = Person("John", "Doe")
person_1.student_summary()
```


Atos

Inheritance for SOLID principle

Liskov substitution solution

- Class interfaces should be clean (intuitive and easy to use)
- Method Interfaces should be the same between parent and child class
- →When overriding a method, the **interface** of the new method should be the same as the original one
  - Same **parameters, return type, preconditions** (substitutable)

```python
class Student(Person):
    def __init__(self, fname, lname, graduation_year):
        super().__init__(fname, lname)
        self.graduation_year = graduation_year

    def printname(self):
        print(
            f"The student {self.firstname} {self.lastname} "
            f"graduated in {self.graduation_year}")


student_1 = Student("Mary", "Sue", 2019)
student_1.printname()

person_1 = Person("John", "Doe")
person_1.printname()
```

```
The student Mary Sue graduated in 2019
John Doe
```

→ Here: Liskov substitution not fulfilled

Inheritance for SOLID principle

Interface segregation

- Better to define multiple (parent) classes with specific purposes instead of a single general purpose class
- → improves code consistency and maintainability
- The class **interface** (its purpose and method structure) **should be specific and segregated** from the others

Inheritance for SOLID principle

Dependency Inversion

- An important parent class should not import logic from another implemented low-level module
- → can create unnecessary dependencies
- → if the low-level module changes, maybe all inheriting subclasses from the parent class have to be modified

07

Exercise Time

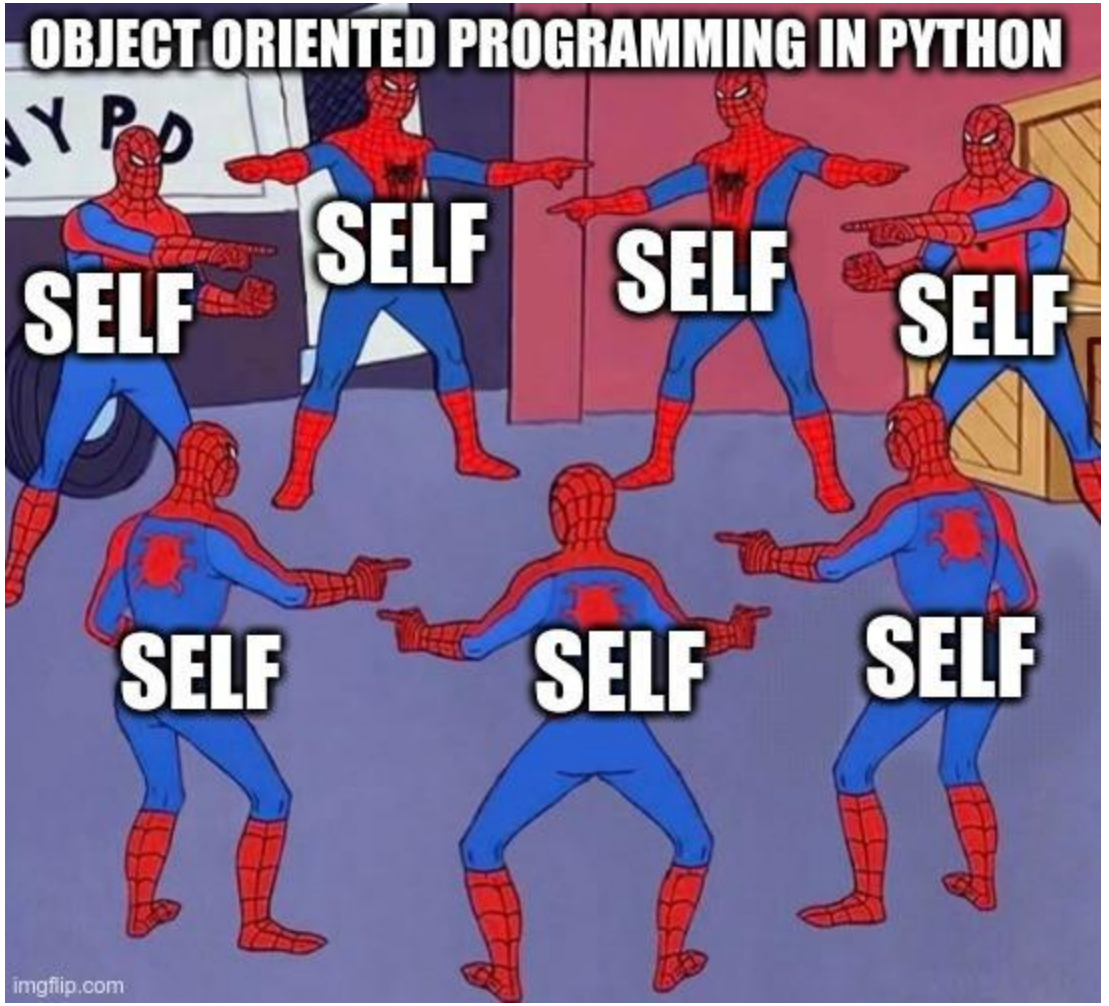
Exercise Time

Vehicle Rental

- Take the three given class files and convert them to a strong inheritance implementation of the parent class Vehicle, following all Best Practices
  - WDSKI25A-Advanced-Programming/Exercise_3 at main · Marius2311/WDSKI25A-Advanced-Programming

- If already finished, create a class Person and subclasses Customer and Vendor, each with needed methods and attributes to realize the rental of a vehicle object

Meme of the day



Next Lecture

1. Coding Guidelines
2. Object Oriented Programming
3. Inheritance
4. GUIs & Interface
5. Code optimization